

Ce code simule l'attribution automatique de fragments d'une vidéo à plusieurs sous-titres. Il gère aussi le cas où un sous-titre se déconnecte, et il réattribue automatiquement ses fragments à quelqu'un d'autre. En plus, à la fin, il enregistre les résultats dans un fichier CSV.

Fragment fictif

Ici, on n'a quatre fragments de vidéo. Chaque fragment a : un id, un temps start (début), un temps end (fin).

```
fragments = [
    {"id": 1, "start": 0.0, "end": 3.5},
    {"id": 2, "start": 3.5, "end": 7.0},
    {"id": 3, "start": 7.0, "end": 10.5},
    {"id": 4, "start": 10.5, "end": 15.0},
]
```

Liste des sous-titres

```
sous_titres = ["Alice", "Bob", "Charlie"]
```

exemple : on a 3 sous-titres

Attribution initiale en Round Robin

```
fragments_state = {}
for i, fragment in enumerate(fragments):
    sous_titre = sous_titres[i % len(sous_titres)]
    fragments_state[fragment["id"]] = {"sous_titre": sous_titre, "statut": "en cours"}
```

C'est une méthode d'attribution cyclique :

1 → Alice
2 → Bob
3 → Charlie
4 → Alice
5 → Bob
... etc.

État des sous-titres

```
sous_titre_statuts = {
    "Alice": "connecté",
    "Bob": "connecté",
    "Charlie": "déconnecté",
}
```

On simule un cas où Charlie est déconnecté. Ça va déclencher la réattribution.

Détection et marquage des fragments à réattribuer

```
for frag_id, info in fragments_state.items():
    if (info['sous_titreur'] == "Charlie" and sous_titreur_statuts["Charlie"] != "connecté"
        and info['statut'] != "terminé"):
        fragments_state[frag_id]['sous_titreur'] = None
        fragments_state[frag_id]['statut'] = "à réattribuer"
```

Ici, dans ce bloc, on parcourt tous les fragments, Si un fragment est attribué à Charlie et que Charlie est déconnecté, Alors :

- Le sous-titreur devient None
- Le statut devient "à réattribuer"

Réattribution aux sous-titreurs connectés

```
for frag_id, info in fragments_state.items():
    if info['statut'] == "à réattribuer":
        for st in sous_titreurs:
            if sous_titreur_statuts[st] == "connecté":
                fragments_state[frag_id]['sous_titreur'] = st
                fragments_state[frag_id]['statut'] = "en cours"
                break
```

Pour chaque fragment à réattribuer, On cherche un sous-titreur connecté, On lui donne le fragment, Puis on fait break → on s'arrête (un fragment = une personne)

Affichage du résultat

```
print("Simulation attribution dynamique après déconnexion :")
for frag_id, info in fragments_state.items():
    print(f"Fragment {frag_id} → {info['sous_titreur']}, statut : {info['statut']}")
```

Ça affiche l'attribution finale dans le terminal.
sortie:

```
In [1]: runfile('C:/Users/marie/STC/
assemblage.py', wdir='C:/Users/marie/STC')
Simulation attribution dynamique après
déconnexion :
Fragment 1 → Alice, statut : en cours
Fragment 2 → Bob, statut : en cours
Fragment 3 → Alice, statut : en cours
Fragment 4 → Alice, statut : en cours
```

Sauvegarde dans un fichier CSV

```

with open("attributions_finales.csv", "w", newline="", encoding="utf-8") as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(["FragmentID", "Sous-titreur", "Statut"])
    for frag_id, info in fragments_state.items():
        writer.writerow([frag_id, info['sous_titreur'], info['statut']])

```

Ce bloc crée un fichier attributions_finales.csv et ajoute une ligne par fragment :

	A	B	C	D	
1	FragmentID,Sous-titreur,Statut				
2	1,Alice,en cours				
3	2,Bob,en cours				
4	3,Alice,en cours				
5	4,Alice,en cours				
6					

Conclusion

Le code attribue les fragments aux sous-titreurs avec la méthode Round Robin

- ♦ Il vérifie si un sous-titreur est connecté
- ♦ S'il est déconnecté, ses fragments sont marqués "à réattribuer"
- ♦ Puis ils sont réattribués à un sous-titreur encore connecté
- ♦ Le résultat est affiché et sauvegardé dans un fichier CSV